

INFORMAÇÕES

MÓDULO	AULA	INSTRUTOR
08 ASP.NET Core Web API - Avançado - EF Core, LINQ e Segurança	01 - LINQ e Segurança inicial da API	Guilherme Paracatu

1. Dominando Consultas Complexas

🎯 **Objetivo:** Compreender o funcionamento interno do LINQ (Language Integrated Query) e a tradução de expressões C# para SQL de alta performance. Vamos dominar a montagem dinâmica de consultas, agregações e o conceito de projeção de dados.

1.1 A Teoria da Execução Adiada (Deferred Execution)

Um dos conceitos mais incompreendidos é achar que o LINQ vai ao banco de dados assim que a variável é declarada. O LINQ, na verdade, opera sob o princípio da **Execução Adiada**.

Quando você escreve um `.Where()` ou um `.OrderBy()`, você não está executando uma busca; você está apenas **construindo uma frase SQL na memória do C#**. A ida ao banco de dados (a "Materialização" da consulta) só acontece quando você invoca um comando de execução final.

- **Comandos de Construção (Não vão ao banco):** `.Where()`, `.OrderBy()`, `.Select()`, `.GroupBy()`, `.Take()`, `.Skip()`. Eles retornam um `IQueryable`.
- **Comandos de Materialização (Disparam o SQL):** `.ToList()`, `.ToArray()`, `.First()`, `.FirstOrDefault()`, `.Single()`, `.Count()`, `.Any()`.

O Poder da Construção Dinâmica: Graças à execução adiada, podemos montar consultas em etapas, ideal para telas com múltiplos filtros de pesquisa opcionais:

C#

```
// 1. Inicia a construção da query (Nenhum SQL foi executado ainda)
var query = _context.Veiculos.AsQueryable();

// 2. Se o usuário preencheu a marca, adiciona o filtro na frase SQL
if (!string.IsNullOrEmpty(filtroMarca))
{
    query = query.Where(v => v.Modelo.Marca.Nome == filtroMarca);
}

// 3. Se preencheu o preço máximo, adiciona mais um filtro
if (precoMaximo.HasValue)
```

```
{
    query = query.Where(v => v.Preco <= precoMaximo.Value);
}
```

```
// 4. SÓ AGORA o EF Core traduz tudo para SQL e vai ao banco!
var resultadoFinal = await query.ToListAsync();
```

1.2 Agrupamento e Estatísticas (GroupBy)

Muito usado para Dashboards e Fechamentos. O `.GroupBy()` divide nossa lista de dados do banco em "baldes" (grupos) baseados em uma chave específica. Após agrupar, podemos aplicar funções de agregação (Sum, Average, Count, Max, Min) em cada balde.

```
C#
var estatisticasPorMarca = await _context.Veiculos
    .GroupBy(v => v.Modelo.Marca.Nome) // O balde é o Nome da Marca
    .Select(grupo => new
    {
        Marca = grupo.Key, // A chave do balde
        TotalVeiculos = grupo.Count(),
        PatioTotal = grupo.Sum(v => v.Preco),
        PrecoMedio = grupo.Average(v => v.Preco)
    })
    .ToListAsync();
```

2. Segurança Básica de APIs: Protegendo as Portas do Backend

🎯 **Objetivo:** Compreender que uma API exposta na internet é um alvo constante. Vamos estudar e implementar as barreiras vitais de proteção: Gerenciamento seguro de senhas (User Secrets), Controle de Origem (CORS) e Autenticação de Sistemas via API Key.

2.1 A Criptografia em Trânsito (HTTPS)

O HTTP padrão envia dados em texto puro. Se alguém interceptar a rede (um ataque *Man-in-the-Middle* em um Wi-Fi público), verá as senhas, os tokens e os dados dos clientes da sua API.

A primeira camada de segurança de qualquer API é o HTTPS (Hyper Text Transfer Protocol Secure). Ele criptografa o tráfego usando certificados TLS. Para quem intercepta, os dados parecem uma "sopa de letrinhas" indecifrável.

No ASP.NET Core, utilizamos o middleware `app.UseHttpsRedirection();` no arquivo `Program.cs`. A função dele é simples: se um cliente tentar acessar a API de forma insegura via `http://`, o servidor bloqueia e o redireciona obrigatoriamente para a versão segura `https://`.

No entanto, para que o HTTPS funcione perfeitamente no ambiente de desenvolvimento local (especialmente usando VS Code), precisamos dominar três regras arquiteturais:

🚧 **Regra 1: A Ordem do Pipeline (O Porteiro)** O pipeline do .NET é lido de cima para baixo. O redirecionamento de segurança deve atuar como o primeiro "porteiro" da aplicação. Se você colocar o Swagger antes do redirecionamento, o Swagger vai devolver a tela HTML e encerrar a requisição (curto-circuito) antes mesmo da regra do HTTPS ser lida!

C#

```
// FORMA CORRETA NO PROGRAM.CS:
app.UseHttpsRedirection(); // <- 1º Ele barra quem não tem HTTPS

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();          // <- 2º Só depois ele entrega a documentação
    app.UseSwaggerUI();
}
```

⚙️ **Regra 2: O Perfil de Inicialização (launchSettings.json)** Para que o redirecionamento funcione, o seu servidor local (Kestrel) precisa estar com a porta segura "ligada". Ao rodar projetos no VS Code, às vezes ele inicia usando o perfil "http" básico. Se a sua API não estiver redirecionando, abra o arquivo Properties/launchSettings.json e exclua o bloco do perfil "http", deixando apenas o bloco "https". Assim, você obriga o servidor a subir as duas portas simultaneamente (ex: 7125 segura e 5228 insegura).

🔒 **Regra 3: A Tela Vermelha de Certificado Inválido (Troubleshooting)** Ao rodar a API com HTTPS pela primeira vez, o navegador (Chrome/Edge) exibirá uma tela vermelha assustadora com o erro: *"Sua conexão não é particular"*.

- Por que isso acontece? No ambiente local (localhost), o .NET cria um certificado "autoassinado" (falso) apenas para testes. Como o Google Chrome não conhece quem criou esse certificado, ele bloqueia o acesso por precaução.
- A Solução Definitiva: Pare a sua API, abra o terminal do VS Code e avise o seu Windows para confiar nesse certificado de desenvolvimento rodando o comando:

Bash

dotnet dev-certs https --trust

(Uma janela do Windows aparecerá pedindo confirmação. Clique em "Sim". Feche o navegador, rode a API novamente e veja o lindo cadeado cinza de segurança aparecer!)

- A Solução Quebra-Galho: Na mesma tela vermelha de erro, clique no botão "Avançado" (ou "Ocultar detalhes") e clique no link menor que diz: "Ir para localhost (não seguro)". O Chrome vai criar uma exceção manual e abrir o seu Swagger.

2.2 O Cofre do Desenvolvedor: User Secrets no VS Code

A Regra de Ouro: NUNCA, em hipótese alguma, coloque senhas de banco de dados, chaves de API ou tokens dentro do arquivo appsettings.json e comite isso no GitHub. Existem bots varrendo repositórios públicos 24 horas por dia em busca de credenciais vazadas.

Para proteger suas senhas no ambiente de desenvolvimento local, o .NET possui uma ferramenta chamada **User Secrets**. Ela salva suas credenciais em uma pasta oculta no seu sistema operacional (fora da pasta do projeto), garantindo que o Git nunca as veja.

Tutorial: Configurando User Secrets pelo Terminal do VS Code

1. Inicialize o cofre secreto: No terminal do VS Code, na raiz do seu projeto API, rode:

Bash

```
dotnet user-secrets init
```

(Isso cria um ID único no seu arquivo .csproj para vincular o projeto ao cofre da sua máquina).

2. Guarde os seus segredos (A Chave de API e o Banco de Dados): Agora vamos tirar as informações sensíveis do código. Rode os seguintes comandos um por vez:

Para a Chave de API:

Bash

```
dotnet user-secrets set "Seguranca:MinhaApiKey" "SenhaSuperSecreta123!"
```

Para a String de Conexão do PostgreSQL: *(Note o uso da palavra ConnectionStrings antes dos dois pontos. Isso é um padrão do .NET!)*

Bash

```
dotnet user-secrets set "ConnectionStrings:ConexaoPadrao"
```

```
"Host=localhost;Database=DbConcessionaria;Username=postgres;Password=SenhaDoSeuBancoAqui"
```

3. Como usar no C#? A mágica do ASP.NET Core é que você não precisa mudar a estrutura do seu código! O framework é inteligente o suficiente para procurar primeiro no appsettings.json; se não achar (ou se o valor do cofre for mais recente), ele busca automaticamente no seu *User Secrets*.

- **Para ler a API Key:**

C#

```
var minhaChave = builder.Configuration.GetValue<string>("Seguranca:MinhaApiKey");
```

- **Para ler o Banco de Dados (O Jeito Sênior):** Graças ao prefixo ConnectionStrings que usamos no terminal, você pode usar o atalho nativo do framework:

C#

```
var stringConexao = builder.Configuration.GetConnectionString("ConexaoPadrao");
```

```
// Injetando no DbContext:
```

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseNpgsql(stringConexao));
```

2.3 Entendendo o CORS a Fundo (Cross-Origin Resource Sharing)

O CORS é uma proteção exigida pelos Navegadores de Internet (Chrome, Edge). A "Política de Mesma Origem" dita que um site hospedado em `www.meu-front.com` não tem permissão para fazer requisições para uma API em um endereço diferente (`api.meu-backend.com`). O navegador atua como um "segurança de balada", bloqueando a entrada.

A Política Restrita (Padrão Ouro para Produção + Ambiente Local) Para liberar a comunicação, configuramos Políticas no `Program.cs`, dizendo exatamente em quais origens nós confiamos.

```
C#
builder.Services.AddCors(options =>
{
    options.AddPolicy("PoliticaRestrita", policy =>
    {
        policy.WithOrigins(
            "http://localhost:3000",           // Front-end React Local
            "https://meu-front-oficial.com",   // Front-end Produção
            "https://localhost:5001"          // O próprio Swagger da API (Garante
testes locais sem bloqueio)
        )
        .WithMethods("GET", "POST", "PUT") // Bloqueia DELETE por segurança, se
desejar
        .AllowAnyHeader(); // Permite envio de tokens/chaves no cabeçalho
    });
});
```

⚠ Atenção à Ordem dos Middlewares: O `app.UseCors("PoliticaRestrita");` deve ser colocado **depois** do `UseRouting()` e **antes** do `UseAuthorization()`!

2.4 Protegendo Endpoints com API Key

Se o JWT (JSON Web Token) é usado para identificar *Pessoas* (usuários logados no Front-end), a **API Key** é usada para identificar *Sistemas* (B2B). É um "crachá de acesso" em formato de texto. Se a Locadora parceira quiser enviar carros para a nossa API de forma automatizada, ela enviará uma chave secreta no cabeçalho (Header) da requisição.

1. Criando o Filtro de Segurança (ApiKeyAttribute.cs) Crie uma pasta `Security` e adicione este filtro. Ele interceptará a requisição antes de chegar no Controller.

```
C#
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
public class ApiKeyAttribute : Attribute, IAsyncActionFilter
{
    private const string ApiKeyHeaderName = "X-API-KEY";
```

```

    public async Task OnActionExecutionAsync(ActionExecutingContext context,
    ActionExecutionDelegate next)
    {
        // 1. Verifica se a chave veio no cabeçalho
        if (!context.HttpContext.Request.Headers.TryGetValue(ApiKeyHeaderName, out var
    extractedApiKey))
        {
            context.Result = new UnauthorizedObjectResult("API Key não informada no
    cabeçalho.");
            return;
        }

        // 2. Pega a Chave Verdadeira salva no User Secrets
        var configuration =
    context.HttpContext.RequestServices.GetRequiredService<IConfiguration>();
        var apiKeyVerdadeira = configuration.GetValue<string>("Seguranca:MinhaApiKey");

        // 3. Compara as chaves
        if (!apiKeyVerdadeira.Equals(extractedApiKey))
        {
            context.Result = new UnauthorizedObjectResult("API Key inválida. Acesso
    negado.");
            return;
        }

        // Tudo certo! Deixa a requisição seguir para o Controller.
        await next();
    }
}

```

2. O Escopo da Segurança: Onde colocar o [ApiKey]? Você pode proteger toda a classe ou apenas rotas específicas:

- **Nível de Classe:** Coloque [ApiKey] em cima do public class VeiculosController. Todos os métodos exigirão a chave.
- **Nível de Método:** Coloque [ApiKey] apenas em cima do [HttpPost]. O resto do Controller continuará público.

2.5 Configurando o Swagger para usar a API Key (O Cadeado)

Se nós protegermos a API, o nosso Swagger retornará erro 401 (Unauthorized), pois ele não sabe enviar o cabeçalho. Precisamos ensinar o Swagger a exibir o botão "Authorize" (o ícone de cadeado).

No Program.cs, modifique o serviço do Swagger:

```

C#
using Microsoft.OpenApi.Models;

builder.Services.AddSwaggerGen(c =>
{

```

```

c.SwaggerDoc("v1", new OpenApiInfo { Title = "API Concessionária", Version = "v1" });

// Avisa o Swagger que existe um Header exigido
c.AddSecurityDefinition("ApiKey", new OpenApiSecurityScheme
{
    Description = "Insira a sua API Key secreta no campo abaixo.",
    Name = "X-API-KEY", // Tem que ser exatamente o mesmo nome do filtro!
    In = ParameterLocation.Header,
    Type = SecuritySchemeType.ApiKey,
    Scheme = "ApiKeyScheme"
});

// Aplica a exigência globalmente na interface
c.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
        new OpenApiSecurityScheme
        {
            Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme,
Id = "ApiKey" },
            Scheme = "oauth2",
            Name = "ApiKey",
            In = ParameterLocation.Header,
        },
        new List<string>()
    }
});
});
});

```

2.5 Criando "Rotas de Fuga": O Atributo [AllowAnonymous]

Até agora, nossa API é um castelo totalmente trancado. Mas e se o marketing quiser que a lista de veículos seja pública para atrair clientes, enquanto o cadastro de novos carros continua exigindo a chave?

Para isso, usamos o atributo nativo [AllowAnonymous]. Porém, como criamos um **Filtro Customizado** (ApiKeyAttribute), precisamos ensinar o nosso filtro a respeitar essa regra.

1. Atualizando o Filtro (ApiKeyAttribute.cs)

Abra o seu filtro e adicione esta verificação logo no início do método. Ela serve para o filtro dar uma "espiada" no método antes de barrar a entrada:

```

C#
public async Task OnActionExecutionAsync(ActionExecutingContext context,
ActionExecutionDelegate next)
{
    // --- REGRA DE OURO: VERIFICAÇÃO DE ANÔNIMOS ---
    // Verifica se o método ou a classe possui o atributo [AllowAnonymous]
    var permiteAnonimos = context.ActionDescriptor.EndpointMetadata
        .Any(em => em.GetType() == typeof(AllowAnonymousAttribute));
}

```

```

    if (permiteAnonimos)
    {
        await next(); // Se permitir anônimos, pula a validação e segue o baile!
        return;
    }
    // -----

    // O restante do código de validação da API Key continua aqui embaixo...
    if (!context.HttpContext.Request.Headers.TryGetValue(ApiKeyHeaderName, out var
extractedApiKey))
    {
        context.Result = new UnauthorizedObjectResult("API Key não informada.");
        return;
    }
    // ...
}

```

2. Como usar no Controller

Agora você tem o melhor dos dois mundos. Você pode trancar a classe inteira e abrir apenas "janelas" específicas:

```

C#
[ApiController]
[Route("api/[controller]")]
[ApiKey] // Tranca tudo por padrão
public class VeiculosController : ControllerBase
{
    [HttpGet]
    [AllowAnonymous] // ESTA ROTA É PÚBLICA: Qualquer um pode ver o catálogo!
    public async Task<IActionResult> Get() { ... }

    [HttpPost]
    // SEM AllowAnonymous: Só entra quem tiver o "ITB-Token" no cabeçalho!
    public async Task<IActionResult> Post([FromBody] VeiculoDto dto) { ... }
}

```

 **Explicação Técnica:** O comando `context.ActionDescriptor.EndpointMetadata` é o que chamamos de **Reflexão em Tempo de Execução**. O .NET olha para o método que está prestes a ser chamado e verifica se ele tem a "etiqueta" `[AllowAnonymous]`. Se tiver, o nosso filtro entende que deve abrir a porta e não pedir o crachá (API Key).

🔒 Desafio Prático: Ajustes de Segurança ITB

Cenário: A API da concessionária está quase pronta, mas o supervisor pediu alguns ajustes rápidos de segurança para facilitar a integração com o time de marketing e o suporte técnico.

🔑 Etapa 1: Chave de Suporte (User Secrets)

O time de suporte precisa de um endpoint para verificar se a API está online, mas essa chave não pode ficar exposta no código.

- **Ação:** No terminal, crie um novo segredo: `dotnet user-secrets set "Seguranca:ChaveSuporte" "ITB2026"`
- **O Desafio:** No seu `VeiculosController`, crie um método simples: `[HttpGet("check")]`.
- **Raciocínio:** Use o `IConfiguration` para ler essa chave. O método deve retornar `Ok("API Online")` somente se o usuário enviar um Header chamado `Suporte-Token` com o valor correto.

🔓 Etapa 2: Liberando o Site do Parceiro (CORS)

Uma empresa parceira vai exibir nossos carros em um portal deles que roda em `http://localhost:8080`.

- **Ação:** No `Program.cs`, localize a sua política de CORS (`"PoliticaRestrita"`).
- **O Desafio:** Adicione essa nova URL (`http://localhost:8080`) na lista de origens permitidas (`WithOrigins`).
- **Raciocínio:** Agora a sua política deve aceitar **duas** origens: a da aula (`localhost:3000`) e a do novo parceiro.

🔑 Etapa 3: Renomeando o "Crachá" (Refatoração)

O TI pediu para mudar o nome do header de segurança para algo que identifique a empresa.

- **Ação:** No arquivo `ApiKeyAttribute.cs`, mude o nome do header de `X-API-KEY` para `ITB-Token`.
- **O Desafio:** Após mudar no filtro, o **Swagger** vai parar de funcionar (o cadeado vai enviar o nome antigo).
- **Raciocínio:** Você deve ir no `Program.cs`, na configuração do Swagger, e atualizar o `Name` para `ITB-Token` para que o cadeado volte a funcionar.

📄 Etapa 4: Vitrine Aberta (`AllowAnonymous`)

Atualmente, a Controller inteira exige a chave de API. Mas o marketing quer que a listagem de veículos seja pública para qualquer um ver no site.

- **Ação:** No `VeiculosController`, mantenha o `[ApiKey]` no topo da classe, mas adicione o atributo `[AllowAnonymous]` no método `Get()` (o que lista todos os veículos).
- **O Desafio:** Teste no Swagger: o método de listar deve funcionar mesmo sem você clicar no cadeado (`Authorize`). Já o método de Post deve continuar exigindo a chave.
- (Objetivo: Praticar a criação de exceções em rotas protegidas).

